

Largest image

The world's largest giga-pixel image is made of 1,655 21-MP images

ASUS 1201N

ASUS's new Atom netbook is powered by the NVIDIA Ion platform

MySQL 5.5

New beta of MySQL 5.5 codenamed "Betony" released

be of 19x19 pixels. Since this will be used as an icon on the toolbar, it is best if it's a PNG file with a transparent backdrop, ensuring that it will blend with any theme that is applied to Chrome.

popup

In this file, we will actually add the code which will load and display the RSS feed contents. To load the RSS feed, we use the `XMLHttpRequest` object which is commonly used in all AJAX applications for loading data onto a page asynchronously.

```
var req = new XMLHttpRequest();
req.open("GET", "http://www.thinkdigit.com/rss/tech-news", true);
req.onload = showRssFeed;
req.send(null);
```

First, we create a new `XMLHttpRequest` object which will then be used to load the feed contents. This object is stored in the `req` variable.

The second line actually instructs the `XMLHttpRequest` object that it is supposed to retrieve the feed located at `http://www.thinkdigit.com/rss/tech-news`, which is the `thinkdigit.com` feed location.

In the third line, we specify the function that needs to be called when the feed is done loading. Here, we create a function called `showRssFeed` to use the downloaded feed content to generate a list of links.

The fourth line simply sends the `GET` request to the specified URL and begins the loading process.

Now we look at the `showRssFeed` function that will do the actual work of rendering the feed contents. However, before we do that, let us briefly go over the structure of RSS.

An RSS feed is simply an XML file with its own syntax of tags. Details of each story in an RSS feed are encapsulated in an `<item>` tag. The `<item>` tag contains further tags which give details of each story. The `<item>` tag contains (listing only the important ones used in the example):

- `<title>` tag that contains the title of the story.
- `<link>` tag that contains a link to the

original article.

- `<description>` tag that contains an extract (or the entirety) of the article (not used in this example).

```
function showTDRssFeed() {
    var posts = req.responseXML.
    getElementsByTagName("item");
    document.body.
    appendChild(document.
    createElement("hr"));
    for (var i = 0; i<5; i++) {
        var story = document.
        createElement("div");
        story.class = "storylink";
        var storyLink = document.
        createElement("a");
        storyLink.href = posts[i].
        getElementsByTagName("link")[0].
        childNodes[0].nodeValue;
        storyLink.innerHTML =
        HTML = posts[i].
        getElementsByTagName("title")
        [0].childNodes[0].nodeValue;
        story.
        appendChild(storyLink);
        document.body.
        appendChild(story);
        document.body.
        appendChild(document.
        createElement("hr"));
    }
}
```

```
var posts = req.responseXML.
getElementsByTagName("item");
```

In the first line in the function, we use the `responseXML` property of `req`, the `XMLHttpRequest` object (that contains the XML content loaded from the RSS feed). The `responseXML` property contains the XML structure of the RSS feed.

You can reload the extension after making even a small change, and see how it affects your extension

Now by running the `getElementsByTagName` function we get an array of all the items in the feed into the array named `posts`.

```
for (var i = 0; i<5; i++) {
    Let us leave the second line for now
```

and go to the for loop. The for loop is just repeating the operating in it 5 times, to display the first 5 stories in the RSS feed. We will here iterate over each item in the `posts` array.

```
var story = document.
createElement("div");
story.class = "storylink";
```

In these lines, we are creating a new `<div>` element which will be containing our story link. The `<div>` element is being given a class of `storylink` so that we can later style it using CSS.

```
var storyLink = document.
createElement("a");
```

Now we create the actual link to the story using the `<a>` element.

```
storyLink.href =
posts[i].
getElementsByTagName("link")[0].
childNodes[0].nodeValue;
storyLink.innerHTML =
posts[i].
```

```
getElementsByTagName("title")
[0].childNodes[0].nodeValue;
```

This are two rather long lines of code which do the simple task of extracting the link and title from the feed story item and injecting them into the `<a>` element. Lets break it down to get a better idea.

The `posts[i]` is the *i*-th element of the `posts` array, from this feed item, we need to extract the link and the title. We extract these by using the `getElementsByTagName` function with the required tag name as parameter. Since this will return an array of link and title elements (even though it will contain only one object), we use the first element of the array using `[0]`. This will return us something like (for the first `<link>` tag):

```
<link>http://www.thinkdigit.com/General/Soccer-Dance-Kung-Fu-AcYut-does-_3822.html</link>
```

Since we only need the link itself from this XML element, we use the `childNodes` property (which will again be an array of child nodes although only with one text element containing the link) and access the first child by appending `[0]`. This will now finally give us a string with the link. We now use the value of that string using the `nodeValue` property and get